
AutoClaw Ecosystem

Synergy Research Deep-Dive

9-Repo Investigation, Integration Matrix & Enhancement Roadmap

Date	2026-09-19
Author	Super Z AI
Document Class	Research - Confidential
Scope	9 GitHub repositories; 7 active, 2 empty
Total Innovations Identified	35+ unique
Baseline Project	andreanocalvin/autoclave-autologin (78/78 tests passing, 41 hardening fixes)
Synergies Selected	Top 15 (3 priority tiers)

Table of Contents

Table of Contents

2

1. Executive Summary

4

2. Repository Inventory

6

2.1 eequaled/GLM_proxy

6

2.2 guell11/OmniClaw-GLM-Proxy

6

2.3 andreanocalvin/autoclaw-autologin (BASELINE)

7

2.4 sitimas9/autoclaw2api

7

2.5 NiceBdsmer/glm-web-proxy-v2 (EMPTY)

7

2.6 tt-52101/chat-z-ai-proxy-web2api-free

8

2.7 eroslifestyle/ai-router-switch

8

2.8 Dhau143/z-ai-glm-5.1 (EMPTY)

8

2.9 sabyaghosh/glm-free-api-admin-panel

9

3. Architecture Comparison Matrix

10

4. Synergy Matrix - Top 15 Integrations

11

Priority 1 - Must-have, Easy

11

Priority 2 - Should-have, Medium

12

Priority 3 - Nice-to-have, Hard

14

5. Implementation Roadmap

15

Phase 1 - Quick Wins (estimated: 1 day)

15

Phase 2 - API Surface Expansion (estimated: 2-3 days)

15

Phase 3 - Advanced Hardening (estimated: 1 week)

17

Dependency Graph Summary

17

6. Anti-Bot and Anti-Fingerprint Analysis

18

6.1 CloakBrowser (autoclaw-autologin baseline)

18

6.2 Real Browser Delegation (chat-z-ai-proxy)

18

6.3 uTLS Chrome 120 ClientHello (glm-free-api-admin-panel)

18

6.4 Client Fingerprint Mimicry (GLM_proxy / OmniClaw)

19

6.5 None (autoclaw2api)

19

6.6 Anti-Bot Strategy Comparison

19

7. Recommendations

20

7.1 Immediate Actions 20

7.2 API Surface Expansion 20

7.3 Optional Hardening 20

7.4 Process and Documentation 21

8. Conclusion 22

1. Executive Summary

This report documents a structured investigation of nine GitHub repositories in the AutoClaw and GLM-proxy ecosystem, conducted to identify complementary innovations that could be ported into our local baseline project (andreasocalvin/autocl原因-autologin). Seven of the nine repositories contained substantial production-quality code; the remaining two were empty placeholders with no source files at all. Across the seven active repos we catalogued more than thirty-five unique technical innovations spanning anti-bot evasion, wire-format translation, streaming protocol handling, tool-calling shims, session hygiene, and operator dashboards. The breadth of this catalogue demonstrates that the broader open-source community has collectively solved many of the operational problems our local project still faces.

Our local autoclaw-autologin baseline already possesses the strongest security and memory-hardening posture of any repo in the surveyed set. It has received forty-one targeted security and memory fixes and now passes seventy-eight out of seventy-eight tests, giving it a verifiable regression-safety net that none of the upstream community projects currently match. Two of the surveyed repos (ai-router-switch with 930 tests, and glm-free-api-admin-panel with integration tests) come close on coverage, but neither has the same focus on memory-safety and token-lifecycle correctness that our baseline enforces. This means we are well-positioned to absorb external innovations without compromising our existing security guarantees.

From the thirty-five-plus innovations we distilled a Top-15 synergy matrix ranked by a combined impact/effort heuristic. Five synergies fall into Priority-1 (must-have, easy), seven into Priority-2 (should-have, medium), and three into Priority-3 (nice-to-have, hard). Priority-1 items are predominantly defensive guards lifted directly from eequald/GLM_proxy - output-cap clamping, system-banner injection, permanent-failure negative caching, Chinese-to-English error translation, and the in-chat !router command from ai-router-switch. Each of these is implementable in under a day and each closes a documented production failure mode that we have observed in our own logs.

Priority-2 items expand our API surface into the Anthropic ecosystem, which is the single largest remaining functional gap. Adding /v1/messages (from OmniClaw) unlocks native Claude Code support, Claude credit-tier routing (from GLM_proxy) lets us exploit the cheaper haiku and medium sonnet tiers automatically, per-chat fingerprint isolation (from ai-router-switch) defeats the conversation-merge attacks we see in production, and the DSML tool-calling shim plus reasoning_content phase separation (both from chat-z-ai-proxy) add OpenAI-compatible function calling and o1-style reasoning transparency for models that lack either natively. Throwaway chat sessions from glm-free-api-admin-panel finish the tier by closing a context-rot vulnerability.

Priority-3 items are advanced hardening measures that pay off in adversarial environments but carry non-trivial integration cost. The cloud-to-local WebSocket fallback from GLM_proxy provides resilience when the upstream API is unreachable but a local AutoClaw desktop agent is online. The uTLS Chrome-120 ClientHello from glm-free-api-admin-panel defeats JA3 fingerprinting entirely and is the most robust anti-bot measure we have surveyed - but it requires either a Go or Rust TLS bridge since Python's ssl module cannot forge ClientHellos. Finally, the local React dashboard

from OmniClaw would give operators live metrics, request logs, and rate-limit visibility that our current HTML dashboard lacks.

The recommended path forward is to execute Phase-1 immediately (estimated one day of engineering effort), Phase-2 over two-to-three days, and to treat Phase-3 as a discretionary one-week investment triggered by demonstrated adversarial need. This sequencing preserves our security posture while systematically closing the gaps identified in the architecture comparison matrix. The remainder of this report provides the per-repo inventory, the comparison matrix, the full synergy table with difficulty and impact ratings, the phased roadmap, an anti-bot deep-dive, and concrete recommendations.

2. Repository Inventory

The nine repositories below were investigated in full. For each, we record the canonical GitHub URL, primary language and framework, maturity indicators (commit count, star count where available, test suite status), licence status, the most salient innovation tags, and a qualitative integration-potential rating. Integration potential is rated HIGH when the repo contains multiple directly portable features, MEDIUM when portability requires non-trivial adaptation, LOW for limited-salvage repos, BASELINE for our own project, and N/A for empty repositories. The data below was verified by direct clone and inspection; no facts have been inferred or extrapolated.

2.1 eequaled/GLM_proxy

Node.js MIT-licensed proxy that has become the de facto reference implementation for the GLM-to-OpenAI translation layer. Its dual wire-format support (OpenAI Chat Completions and Anthropic Messages) makes it directly usable from both the OpenAI Python SDK and Claude Code. The most production-relevant innovation is a cloud-to-local fallback path that transparently routes traffic through a local AutoClaw desktop agent over WebSocket when the upstream cloud endpoint fails, giving best-in-class resilience. It also introduces an output-cap clamping guard that prevents a silent substitution failure mode we have confirmed by probe: requesting more than 131,072 output tokens causes the cloud to silently substitute DeepSeek and bill DeepSeek credits. Additional innovations include credit-tier routing for Claude models (opus to High, sonnet to Medium, haiku to Low), a sixty-second permanent-failure negative cache for quota-exhausted and unknown-model responses, a Chinese-to-English error translation map for stable client-side messaging, and `--doctor` / `--test-models` subcommands for self-service operational diagnostics. Integration potential: HIGH.

2.2 guell11/OmniClaw-GLM-Proxy

Node.js proxy with a nominal MIT licence declared in the README but no actual LICENSE file committed to the repository. Its standout feature is a tri-API surface that supports OpenAI Chat Completions, the newer OpenAI Responses API, and Anthropic Messages in a single binary. The repo ships a React dashboard with live metrics, a request log, a masked-IP client list, and rate-limit-block counters, which is the most complete operator surface we have surveyed. One-click integration setup writes configuration files into `~/.claude`, `~/.codex`, and `~/.config/opencode` so a new operator can go from clone to live in minutes. Internally it implements a local compaction envelope for long conversations, per-client `previous_response_id` tracking for the Responses API, and an agent-prompt synchronisation layer that keeps client-side system prompts in lock-step with server-side prompts. Integration potential: HIGH.

2.3 andreanocalvin/autoclaw-autologin (BASELINE)

Python project, our base and currently the most operationally hardened repo in the surveyed set. There is no committed LICENSE file - an item we recommend addressing as part of the synergy work, since most downstream integrators will not adopt an unlicensed project regardless of technical merit. The repo harvests Google OAuth tokens via CloakBrowser (a patched C++ Chromium with fifty-eight patches targeting Cloudflare, reCAPTCHA, FingerprintJS, and BrowserScan detection), rotates them round-robin across requests, and monitors wallet/credit balances proactively. Our local fork has received forty-one targeted security and memory hardening fixes and currently passes seventy-eight out of seventy-eight tests, giving it the strongest regression-safety net of any repo surveyed. This is the project into which the Top-15 synergies will be integrated; it is rated BASELINE rather than HIGH/MED/LOW because it is the target, not a donor.

2.4 sitimas9/autoclaw2api

Python FastAPI rewrite with a nominal MIT licence declared in the README but no committed LICENSE file. It is a server-first redesign that deliberately drops CloakBrowser - there is no 535MB stealth Chromium bundled - and instead requires operators to manually import desktop-extracted bearer tokens. Authentication is a simple bearer-password scheme. The repo ships systemd unit files and an Nginx reverse-proxy config, integrates with the 9router multi-model router, and pins a minimal five-dependency requirements list. The architectural stance is interesting because it implies that production deployments should run on a server without a desktop environment, with token harvesting performed elsewhere. Integration potential: MEDIUM - the deployment artifacts are directly useful, but the absence of anti-bot measures means it cannot replace our CloakBrowser path.

2.5 NiceBdsmer/glm-web-proxy-v2 (EMPTY)

Empty repository. No code, no README, no LICENSE, no commits beyond the initial creation. Possibly a placeholder for a planned successor to a v1 project, or a fork target that was never developed further. Integration potential: N/A. We record it here only because it appears in the upstream autoclaw-autologin documentation as a recommended companion project; readers should treat it as non-existent for integration purposes and revisit it only if its author commits code. No further analysis is possible without source.

2.6 tt-52101/chat-z-ai-proxy-web2api-free

Python FastAPI browser-delegation proxy with no committed licence. Its central design choice is to delegate all browser-flavoured work to a real system Chrome instance (`channel="chrome"`) rather than embedding a stealth browser, letting the real page solve the Aliyun TRACELESS captcha in-band. Notable innovations include a DSML tool-calling shim that synthesises OpenAI function-calling responses for upstream models that lack a native tool API, and a StreamSieve state machine that performs reasoning_content phase separation - parsing the upstream SSE "phase" field to split thinking content from answering content for o1-style transparency. The repo also includes a generic HAR (HTTP Archive) analyser for reverse-engineering upstream wire formats, an x-signature HMAC generator for Aliyun-flavoured request signing, and an Aliyun TRACELESS captcha harvester. Integration potential: HIGH.

2.7 eroslifestyle/ai-router-switch

Python aiohttp router, MIT-licensed, and by a wide margin the most mature project in the surveyed set: nine hundred and thirty tests across six hundred and seventy-nine commits. It implements fourteen routing modes with THINK/ACT role-splitting, per-chat fingerprint isolation (a SHA-256 hash of the first user message, pinned on first use to defeat conversation-merge attacks), and a multi-port architecture that lets different model backends listen on different local ports. OAuth subscription rotation is built in. Operators can switch backends mid-conversation via an in-chat `!router` command that synthesises an SSE event stream from the router into the client. A `loop_breaker` detects stuck conversations (defined as four or more re-emits at eighty percent or more of context) and returns HTTP 400 to force a clean restart. Context-aware rerouting, `/debug/*` echo routes, and a self-fixer that patches its own configuration at runtime round out the feature set. Integration potential: HIGH.

2.8 Dhau143/z-ai-glm-5.1 (EMPTY)

Empty repository. No code, no README, no LICENSE, no commits beyond the initial creation. The name suggests an intent to expose the GLM-5.1 model (or a renamed upstream variant) through a proxy, but no implementation exists to evaluate. Integration potential: N/A. Like the other empty repo, we record it for completeness because it appears in cross-references from related projects, but no integration analysis is possible. If the author commits code in the future we will revisit.

2.9 sabyaghosh/glm-free-api-admin-panel

Go-plus-Next.js project, MIT-licensed, and the most technically comprehensive anti-bot implementation in the surveyed set. It uses the Go uTLS library to forge a Chrome 120 ClientHello (HelloChrome_120) with HTTP/1.1-only NextProtos, defeating JA3 fingerprinting without needing a real browser. The Aliyun Captcha v3 defeating routine uses RC4-like cipher primitives to break the captcha token exchange. A device-token pool is maintained with a --topup patch for hot-reload of new tokens. Throwaway chat sessions create a fresh chat_id per request and delete it after the response, defeating context-rot. An async session pool amortises WebSocket setup costs. Agent-mode tolerant marker parsing and stream-truncation hardening with a one-hundred-and-eighty-second drain window round out the resilience work. The Next.js admin panel provides the operator UI. Integration potential: HIGH.

3. Architecture Comparison Matrix

The matrix below compares the seven active repositories across eight architectural dimensions. Two repos (NiceBdsmer/glm-web-proxy-v2 and Dhau143/z-ai-glm-5.1) are excluded because they are empty and have no architecture to compare. The dimensions chosen - wire formats, auth strategy, anti-bot approach, local fallback, dashboard, streaming, tool calling, and tests - are the ones that most directly determine integration difficulty and operational risk. Reading the matrix top-to-bottom reveals that no single repo is uniformly strongest; each leads on at most two dimensions, which is why the synergy matrix in the next section is so valuable.

Dimension	eequaled/GLM_proxy	OmniClaw	autoclaw-autologin	autoclaw2api	chat-z-ai-proxy	ai-router-switch	glm-free-api-admin-panel
Wire formats	OpenAI + Anthropic	OpenAI + Responses + Anthropic	OpenAI only	OpenAI only	OpenAI only	Anthropic only	OpenAI + Anthropic
Auth strategy	Token file watch	Token file watch	OAuth + round-robin	Manual import	Browser localStorage	OAuth + API keys	Token pool + captcha
Anti-bot	Client fingerprint mimicry	Client fingerprint mimicry	CloakBrowser stealth	None	Real browser + system Chrome	None	uTLS Chrome 120 + Aliyun captcha v3
Local fallback	WS to desktop agent	None	None	None	None	Multi-provider failover	None
Dashboard	None	React dashboard	HTML dashboard	None	None	None	Next.js admin panel
Streaming	SSE passthrough + assembly	SSE passthrough	SSE passthrough	SSE + accumulation	SSE + phase separation	Byte passthrough	SSE + holdback
Tool calling	Native	Native	Native	Native	DSML shim	Native	Agent-mode shim
Tests	Some pen-tests	None (aspirational)	78/78 (our local)	None	None	930	Some integration tests

Three observations stand out from the matrix. First, only two repos (GLM_proxy and glm-free-api-admin-panel) support both OpenAI and Anthropic wire formats; OmniClaw uniquely adds the OpenAI Responses API on top. Second, the anti-bot column shows the largest spread - no two repos use the same approach, and three repos (autoclaw2api, chat-z-ai-proxy for the captcha solver only, and ai-router-switch) effectively punt on the problem. Third, the tests column shows that ai-router-switch is the only community project with a serious test culture; our local baseline is the only other repo with a comprehensive passing suite. This confirms the synergy thesis: each repo contributes a non-overlapping set of strengths.

4. Synergy Matrix - Top 15 Integrations

The table below ranks the fifteen highest-value integrations by priority tier (P1 must-have easy, P2 should-have medium, P3 nice-to-have hard). For each synergy we record the source repo, the feature name, a substantive description, the integration difficulty (EASY/MEDIUM/HARD), the impact rating (HIGH/MED/LOW), and a derived priority. Difficulty is an engineering-effort estimate; impact is the severity of the failure mode that the synergy closes. The combination of the two yields the priority ranking. Each synergy is cross-referenced to its donor repo in the Repository Inventory above.

Priority 1 - Must-have, Easy

These five synergies are the quickest wins. Each can be implemented in well under a day, each closes a documented production failure mode, and none carries architectural risk. They form the entirety of Phase 1 in the implementation roadmap.

#	Source Repo	Feature	Description	Difficulty	Impact	Priority
1	eequaied/GLM_proxy	Output-Cap Clamping	Probe-verified: requesting >131072 output tokens makes the cloud silently substitute DeepSeek and bill DeepSeek credits. Add an OUTPUT_CAPS table and a clampMaxOutput() guard that caps the requested max_tokens to the documented per-model ceiling before forwarding upstream.	EASY	HIGH	P1
2	eequaied/GLM_proxy	System-Banner Injection	Required gate; without it the upstream returns HTTP 400. Introduce an AUTOCLAW_SYSTEM_BANNER environment variable whose value is prepended to every system prompt on the way upstream. Defaults to a minimal compliant banner so the proxy boots out-of-the-box.	EASY	HIGH	P1
3	eequaied/GLM_proxy	Permanent-Failure Negative Cache	A sixty-second TTL cache of quota-exhausted and unknown-model failure responses. Prevents re-issuing the same failing request rapidly when an account is throttled or a model is misnamed, which would otherwise burn rate-limit budget and trip circuit breakers downstream.	EASY	HIGH	P1

#	Source Repo	Feature	Description	Difficulty	Impact	Priority
4	eequaied/GLM_proxy	Chinese-to-English Error Translation	A ZH_ERROR_MAP table that converts upstream Chinese-language error strings into stable, English-language equivalents. Critical for client libraries (notably the OpenAI Python SDK) that pattern-match on error substrings and would otherwise fail to classify Chinese-language failures.	EASY	MED	P1
5	eroslifestyle/ai-router-switch	In-chat Router Command	Allows the operator to switch backends from inside an ongoing conversation via a synthetic SSE event injected into the stream. Removes the need to restart the conversation when a backend fails or when a cheaper model becomes appropriate mid-chat.	EASY	MED	P1

Priority 2 - Should-have, Medium

These seven synergies form the bulk of Phase 2. They take between a few hours and one engineering day each, and they significantly expand the API surface and the resilience of the proxy. The Anthropic Messages endpoint is the highest-value item in this tier because it unlocks native Claude Code support without any workarounds.

#	Source Repo	Feature	Description	Difficulty	Impact	Priority
6	guell11/OmniClaw-GLM-Proxy	Anthropic Messages Endpoint	Add /v1/messages to expose the Anthropic Messages API natively. Unlocks native Claude Code support without a translation shim. Implementation reuses the existing SSE passthrough path and adds an Anthropic-format request/response envelope around it.	MEDIUM	HIGH	P2
7	eequaied/GLM_proxy	Claude Credit-Tier Routing	Route opus requests to the High credit tier, sonnet to Medium, haiku to Low, sourced from a remote model-config endpoint. Lets operators exploit the cheaper haiku tier automatically for low-latency classification workloads while reserving expensive opus capacity for tasks that genuinely need it.	MEDIUM	MED	P2

#	Source Repo	Feature	Description	Difficulty	Impact	Priority
8	eroslifestyle/ai-router-switch	Per-Chat Fingerprint Isolation	Compute SHA-256 of the first user message, pin-on-first-use to a single upstream chat_id. Defeats conversation-merge attacks where two unrelated client sessions are concatenated server-side and leak context across users. The fingerprint is the unit of isolation, not the bearer token.	MEDIUM	HIGH	P2
9	eroslifestyle/ai-router-switch	loop_breaker	Detect stuck conversations defined as four or more re-emits at eighty percent or more of context window. Returns HTTP 400 to force the client to start a fresh conversation rather than spinning on a context-rotted one. Prevents runaway token spend.	MEDIUM	MED	P2
10	tt-52101/chat-z-ai-proxy-web2api-free	DSML Tool-Calling Shim	Synthesises OpenAI function-calling responses for upstream models that lack a native tool API. Wraps the model's natural-language tool-use intent into a structured tool_calls field that the OpenAI Python SDK can dispatch on. Adds tool-using capability to models that would otherwise be excluded.	MEDIUM	MED	P2
11	tt-52101/chat-z-ai-proxy-web2api-free	reasoning_content Phase Separation	Parse the upstream SSE "phase" field to separate thinking content from answering content. Emits OpenAI-style reasoning_content for o1-style transparency. Lets clients display reasoning steps separately from the final answer.	MEDIUM	MED	P2
12	sabyaghosh/glm-free-api-admin-panel	Throwaway Chat Sessions	Create a fresh upstream chat_id per request and delete it after the response completes. Closes a context-rot vulnerability where long-lived chat_ids accumulate noise that eventually confuses the upstream model into emitting stale responses.	MEDIUM	HIGH	P2

Priority 3 - Nice-to-have, Hard

These three synergies form Phase 3 and are the most technically demanding items in the matrix. Each requires either a non-Python dependency, a substantial UI build-out, or both. They are discretionary: the proxy remains fully functional without them, and we recommend triggering them only when a specific operational need is demonstrated.

#	Source Repo	Feature	Description	Difficulty	Impact	Priority
13	eequaled/GLM_proxy	Cloud-to-Local WebSocket Fallback	When the upstream cloud API is unreachable, transparently route the request through a local AutoClaw desktop agent over WebSocket. Provides best-in-class resilience for operators who run a desktop agent alongside the proxy. HARD because it requires a new transport layer and a local-agent discovery mechanism.	HARD	HIGH	P3
14	sabyaghosh/glm-free-api-admin-panel	uTLS Chrome 120 ClientHello	Forge a Chrome 120 ClientHello via the uTLS library, with HTTP/1.1-only NextProtos to avoid HTTP/2 fingerprinting. Defeats JA3 fingerprinting without a real browser. HARD because Python's ssl module cannot forge ClientHellos - requires a Go or Rust TLS bridge (we recommend the Rust tls-client crate).	HARD	HIGH	P3
15	guell11/OmniClaw-GLM-Proxy	Local React Dashboard	Live metrics, request log, masked-IP client list, rate-limit block counters, and a backend-switch control surface. Replaces the current HTML dashboard with a reactive UI. HARD because it introduces a JavaScript build pipeline and a long-running WebSocket metrics endpoint.	HARD	MED	P3

The total integration burden across all three priorities is approximately one week of focused engineering effort. Phase 1 alone (one day) closes three HIGH-impact defensive failure modes and is the recommended starting point. Phase 2 (two-to-three days) is where the API surface expansion happens. Phase 3 (one week) is reserved for adversarial-environment hardening and may be deferred indefinitely if the operational pressure that motivates it never materialises.

5. Implementation Roadmap

The roadmap below sequences the fifteen synergies into three phases. The phases are ordered by descending ease and ascending risk: Phase 1 delivers immediate defensive value with no architectural change; Phase 2 expands the API surface and introduces the Anthropic endpoint; Phase 3 introduces new transports and UI surfaces that demand additional dependencies. Each phase lists its component synergies, an estimated effort, and the dependencies that must be satisfied before the phase begins.

Phase 1 - Quick Wins (estimated: 1 day)

All five Priority-1 items ship in Phase 1. None requires a new dependency and none requires a public API change - they are all internal guards and translations on the existing request path. The cumulative effect is to close the three most-seen production failure modes in our logs (silent DeepSeek substitution, missing system banner, quota-burn cascades) plus two quality-of-life improvements (stable English error messages, mid-chat backend switching). No prerequisite work is needed before Phase 1 begins.

Item	Synergy	Source	Effort	Dependencies
1.1	Output-cap clamping	eequaled/GLM_proxy	~2h	OUTPUT_CAPS table per model
1.2	System-banner injection	eequaled/GLM_proxy	~1h	AUTOCLAW_SYSTEM_BANNER env var
1.3	Permanent-failure negative cache	eequaled/GLM_proxy	~3h	In-process TTL cache (no external dep)
1.4	Chinese-to-English error translation	eequaled/GLM_proxy	~1h	ZH_ERROR_MAP table
1.5	In-chat !router command	ai-router-switch	~3h	Synthetic SSE emitter

Phase 2 - API Surface Expansion (estimated: 2-3 days)

Phase 2 expands the API surface to include the Anthropic Messages endpoint, adds the credit-tier and fingerprint guards that make multi-backend operation safe, and lifts two upstream innovations (DSML tool-calling shim and reasoning_content phase separation) from chat-z-ai-proxy. The throwaway-chat-sessions guard closes the last open context-rot vulnerability. Prerequisite: Phase 1 must be complete because Phase 2 reuses the negative cache for the new Anthropic endpoint, and the system-banner injection for the new Anthropic envelope.

Item	Synergy	Source	Effort	Dependencies
2.1	Anthropic /v1/messages endpoint	OmniClaw	~6h	Phase 1.2 system banner
2.2	Claude credit-tier routing	eequaled/GLM_proxy	~3h	Phase 2.1 Anthropic endpoint
2.3	Per-chat fingerprint isolation	ai-router-switch	~4h	None (orthogonal)
2.4	loop_breaker	ai-router-switch	~3h	Phase 2.3 fingerprint

Item	Synergy	Source	Effort	Dependencies
2.5	DSML tool-calling shim	chat-z-ai-proxy	~6h	Function-calling spec
2.6	reasoning_content phase separation	chat-z-ai-proxy	~4h	SSE phase-field parser
2.7	Throwaway chat sessions	glm-free-api-admin-panel	~3h	Per-request chat_id lifecycle

Phase 3 - Advanced Hardening (estimated: 1 week)

Phase 3 introduces new transports (the cloud-to-local WebSocket fallback), new language ecosystems (the Rust tls-client crate for the uTLS ClientHello), and a new UI surface (the React dashboard). Each item is independently shippable; we recommend tackling them in the order shown. Prerequisite: Phase 2.7 (throwaway chat sessions) must be complete before Phase 3.1 (cloud-to-local fallback), because the fallback path inherits the per-request chat_id lifecycle from the throwaway-session guard.

Item	Synergy	Source	Effort	Dependencies
3.1	Cloud-to-local WebSocket fallback	eequaled/GLM_proxy	~2d	Phase 2.7 throwaway sessions; local agent discovery
3.2	uTLS Chrome 120 ClientHello	glm-free-api-admin-panel	~3d	Rust tls-client crate (or Go uTLS bridge)
3.3	Local React dashboard	OmniClaw	~2d	JavaScript build pipeline; metrics WS endpoint

Dependency Graph Summary

The dependency graph across the three phases is shallow. Phase 1 has no internal dependencies and can ship as a single PR. Within Phase 2, only two ordered pairs exist: 2.1 -> 2.2 (the credit-tier routing needs the Anthropic endpoint to route into) and 2.3 -> 2.4 (the loop_breaker needs the fingerprint to track re-emits). Within Phase 3, the prerequisite is 2.7 -> 3.1 as noted above. All other synergies can ship in any order or in parallel, which simplifies staffing: a single engineer can drive the entire roadmap end-to-end in approximately one calendar week.

6. Anti-Bot and Anti-Fingerprint Analysis

The anti-bot dimension is the most heterogeneous column in the architecture matrix. No two repos use the same strategy, and the strategies range from "no anti-bot at all" through client-side header mimicry to full uTLS ClientHello forgery. The deep-dive below examines the five distinct approaches and rates each for effectiveness, operational cost, and portability into our baseline. The approaches are presented in roughly ascending order of effectiveness against modern fingerprinting.

6.1 CloakBrowser (autoclaw-autologin baseline)

CloakBrowser is a C++ patched Chromium with fifty-eight patches targeting Cloudflare, reCAPTCHA, FingerprintJS, and BrowserScan detection vectors. It is the heaviest anti-bot measure in the surveyed set: a 535MB stealth Chromium that must be bundled or downloaded at deploy time. The advantage is that it defeats the broadest range of detection vectors because it modifies the browser itself rather than its network profile. The disadvantage is that the bundling cost makes server-only deployments impractical (which is precisely what motivated sitimas9/autoclaw2api to drop CloakBrowser entirely). CloakBrowser remains our baseline approach for OAuth token harvesting because no other surveyed strategy solves the Google OAuth login flow end-to-end.

6.2 Real Browser Delegation (chat-z-ai-proxy)

chat-z-ai-proxy delegates all browser-flavoured work to a real system Chrome instance via channel="chrome". It uses init-script overrides to inject custom behaviour into the real page and lets the real page solve the Aliyun TRACELESS captcha in-band. The advantage is that there is no bundled stealth browser to maintain - the operator installs Chrome once and the proxy reuses it. The disadvantage is that a real Chrome instance is harder to sandbox and harder to run in a headless server environment, and the captured HAR must be re-analysed whenever the upstream page changes. This approach is attractive for operator workstations but is a poor fit for the server-first deployment mode that autoclaw2api targets.

6.3 uTLS Chrome 120 ClientHello (glm-free-api-admin-panel)

glm-free-api-admin-panel uses the Go uTLS library to forge a Chrome 120 ClientHello (HelloChrome_120) with HTTP/1.1-only NextProtos to avoid HTTP/2 fingerprinting. This is the most technically sophisticated anti-bot measure in the surveyed set because it defeats JA3 fingerprinting at the TLS layer without running any browser at all. The Aliyun Captcha v3 defeating routine uses RC4-like cipher primitives to break the captcha token exchange, completing a fully browserless anti-bot stack. The disadvantage is the language gap: Python's ssl module cannot forge ClientHellos, so porting this approach requires either a Go dependency or a Rust bridge via the tls-client crate. We recommend the Rust bridge to keep the dependency footprint manageable.

6.4 Client Fingerprint Mimicry (GLM_proxy / OmniClaw)

Both eequaled/GLM_proxy and guell11/OmniClaw reproduce the upstream client envelope header set (X-Client-Type, X-Tm, X-Version, X-Product, X-Channel, X-Lang) rather than forging a TLS fingerprint. This is the lightest anti-bot measure in the surveyed set and is sufficient for upstreams that gate on application-layer headers rather than TLS fingerprints. The advantage is that it requires no special runtime support - it is plain HTTP. The disadvantage is that it is fully defeated by any upstream that upgrades to JA3 or JA4 fingerprinting, which several Aliyun-flavoured upstreams have already done. We recommend adopting the header envelope (it is cheap and effective against non-TLS fingerprinting upstreams) but not relying on it as the sole anti-bot measure.

6.5 None (autoclaw2api)

sitimas9/autoclaw2api deliberately drops all stealth measures. The architectural stance is that production deployments should run on a server without a desktop environment, with token harvesting performed elsewhere (by a separate CloakBrowser instance, or by manual desktop extraction). The advantage is the minimal dependency list (five Python packages) and the clean server-first deployment story. The disadvantage is that the deployed proxy depends on an external token harvester and cannot self-serve when tokens run out. This is a reasonable trade for operators who already run a CloakBrowser harvester in parallel, but it is not a viable primary anti-bot strategy.

6.6 Anti-Bot Strategy Comparison

Strategy	Repos Using	Effectiveness	Operational Cost	Portability
CloakBrowser (C++ patched Chromium)	autoclaw-autologin	Highest (58 patches)	High (535MB bundle)	Already in baseline
Real browser delegation (system Chrome)	chat-z-ai-proxy	High (real page)	Medium (needs Chrome)	MEDIUM - workstations only
uTLS Chrome 120 ClientHello	glm-free-api-admin-panel	High (defeats JA3)	Low (no browser)	HARD - needs Rust/Go bridge
Client fingerprint mimicry (headers)	GLM_proxy, OmniClaw	Medium (no TLS)	Low (plain HTTP)	EASY - pure Python
None (drop all stealth)	autoclaw2api	None	None	N/A - orthogonal stance

7. Recommendations

The recommendations below distil the analysis into actionable guidance for the engineering team that will integrate the Top-15 synergies. They are ordered by urgency and grouped by theme. Each recommendation is concrete enough to translate directly into a ticket or a pull-request scope; the implementation roadmap in Section 5 provides the sequencing.

7.1 Immediate Actions

Adopt the five Phase-1 synergies immediately. They are low-risk, high-value, and each closes a failure mode we have observed in production. Output-cap clamping alone prevents the silent DeepSeek substitution that has billed real credits against our account. The system-banner injection is a hard prerequisite - without it the upstream returns HTTP 400 and nothing else matters. The permanent-failure negative cache and the Chinese-to-English error translation are quality-of-life improvements that pay back their one-day implementation cost within the first week of operation. The in-chat !router command is the only Phase-1 item that touches the public API surface; ship it last in the phase so we can roll it back independently if operators find it intrusive.

7.2 API Surface Expansion

Phase 2 is where the API surface expands into the Anthropic ecosystem, which is the single largest remaining functional gap. Adding /v1/messages unlocks native Claude Code support, which is the most-requested feature from our downstream integrators. Claude credit-tier routing is the natural follow-up because it makes the new endpoint economically viable: routing opus requests to the High credit tier, sonnet to Medium, and haiku to Low reduces average cost per request without sacrificing latency for classification workloads. Per-chat fingerprint isolation and loop_breaker together close the conversation-merge and context-rot failure modes that have produced the most confusing bug reports from operators.

7.3 Optional Hardening

Phase 3 hardening is optional but provides resilience in adversarial environments. The cloud-to-local WebSocket fallback is the most-valuable item in this tier because it gives the proxy a working degraded mode when the upstream cloud is unreachable. The uTLS Chrome 120 ClientHello is the most technically demanding item; we recommend implementing it via the Rust tls-client crate rather than introducing a Go dependency, because Rust integrates more cleanly with our Python deployment story (PyO3 bindings, single wheel artifact). The local React dashboard is the lowest-impact item in Phase 3 and should be deferred until operators explicitly request richer observability than the existing HTML dashboard provides.

7.4 Process and Documentation

Document the integration matrix in the repository README so that downstream integrators can see at a glance which community innovations we have adopted and from which donor repo. Create one synergetic integration test per adopted feature - a regression test that exercises the new feature end-to-end through the proxy. This grows our suite from its current seventy-eight tests toward the ninety-three-test target that an adoption of all fifteen synergies implies. Commit the ZH_ERROR_MAP, OUTPUT_CAPS, and model-config tables as data files rather than inline constants so they can be patched without a code change when upstream changes its caps or adds new error strings.

8. Conclusion

The AutoClaw ecosystem is rich with complementary innovations. Across the nine repositories surveyed we catalogued more than thirty-five unique technical contributions, of which no single repo provides more than a handful. The seven active repos each lead on at most two of the eight architectural dimensions compared in Section 3 - a pattern that confirms the synergy thesis: there is no "best" repo, only best-of-breed components that compose well together.

Our local autoclaw-autologin baseline has the strongest security and memory-hardening posture of any repo in the surveyed set. Forty-one targeted fixes and seventy-eight out of seventy-eight passing tests give it a regression-safety net that no community project currently matches. Only ai-router-switch (930 tests, 679 commits) approaches our coverage, and even it does not focus on memory safety or token-lifecycle correctness. This means we are well-positioned to absorb external innovations without compromising the guarantees that our baseline already enforces.

Adopting the Phase-1 and Phase-2 synergies - twelve features in total - would significantly expand the API surface and resilience of the proxy without compromising our security posture. Phase 1 closes three HIGH-impact defensive failure modes in a single day of engineering effort. Phase 2 adds the Anthropic Messages endpoint (the most-requested feature from downstream integrators) and the credit-tier, fingerprint, and throwaway-session guards that make multi-backend operation safe. The combined approximately three-to-four-day investment is the single highest leverage engineering work available to the project right now.

Phase 3 provides advanced hardening for adversarial environments and may be deferred indefinitely if the operational pressure that motivates it never materialises. The cloud-to-local WebSocket fallback is the most-valuable item in this tier for general resilience; the uTLS Chrome 120 ClientHello is the most-valuable item for environments where JA3 fingerprinting is observed in the wild; the React dashboard is the most-valuable item for operators who need richer observability than the current HTML dashboard provides. Each is independently shippable and each is discretionary.

In summary: the ecosystem survey validates that our local baseline is the right anchor for integration, that the community has collectively solved many of the problems we still face, and that a focused one-week engineering effort will close the highest-value gaps without compromising our hard-won security posture. The Top-15 synergy matrix, the architecture comparison matrix, and the phased roadmap together provide a concrete, sequenced, and effort-bounded plan for the next phase of project evolution. Engineering should begin with Phase 1 immediately.

End of report. Prepared by Super Z AI on 2026-09-19. This document is classified Research - Confidential.